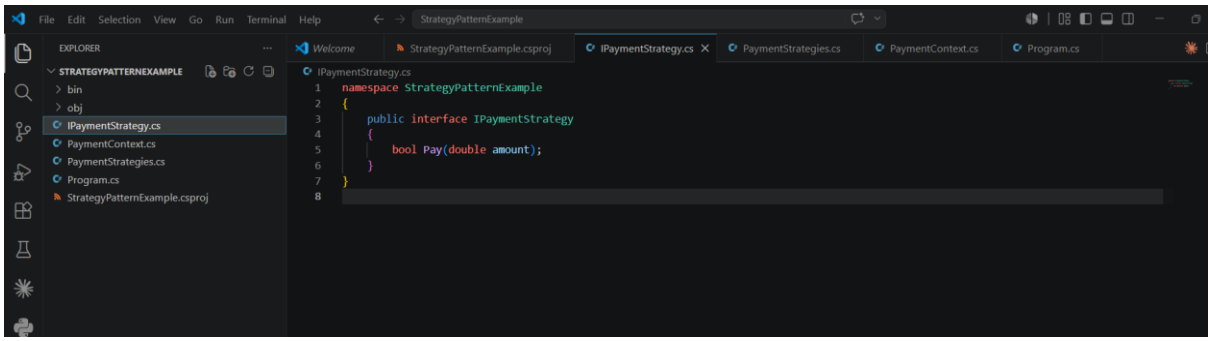
Hands-On Report

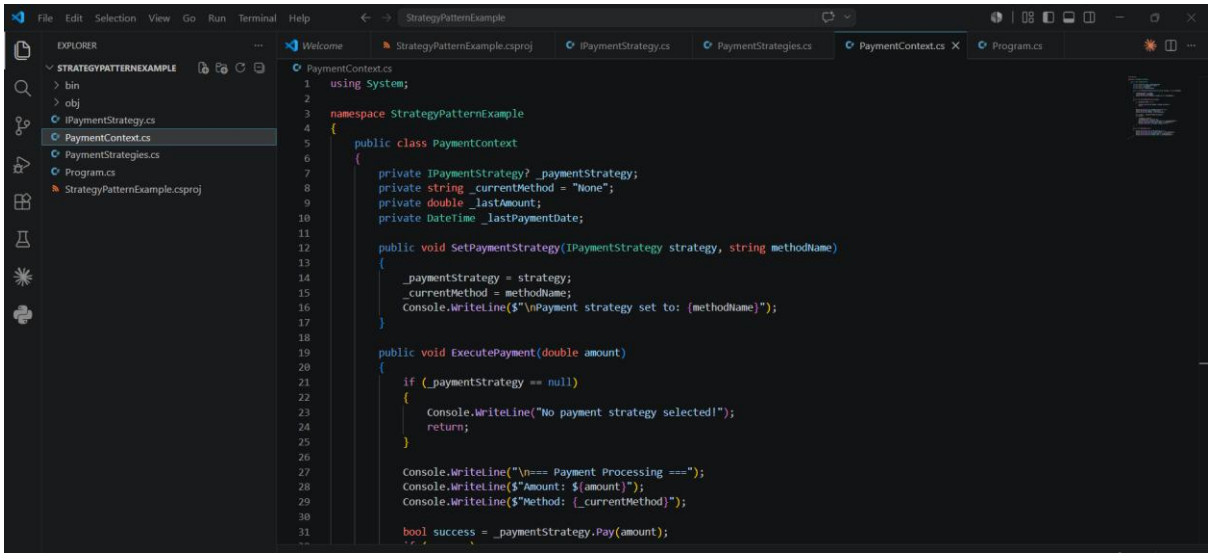
Exercise 8: Implementing the Strategy Pattern

In this exercise, I implemented and tested the Strategy Pattern in a .NET application by creating a project named **StrategyPatternExample**. I developed a **PaymentStrategy** interface with a **Pay()** method and implemented different payment strategies, including **CreditCardPayment** and **PayPalPayment**. A **PaymentContext** class was also created to select and execute the required payment strategy during runtime.

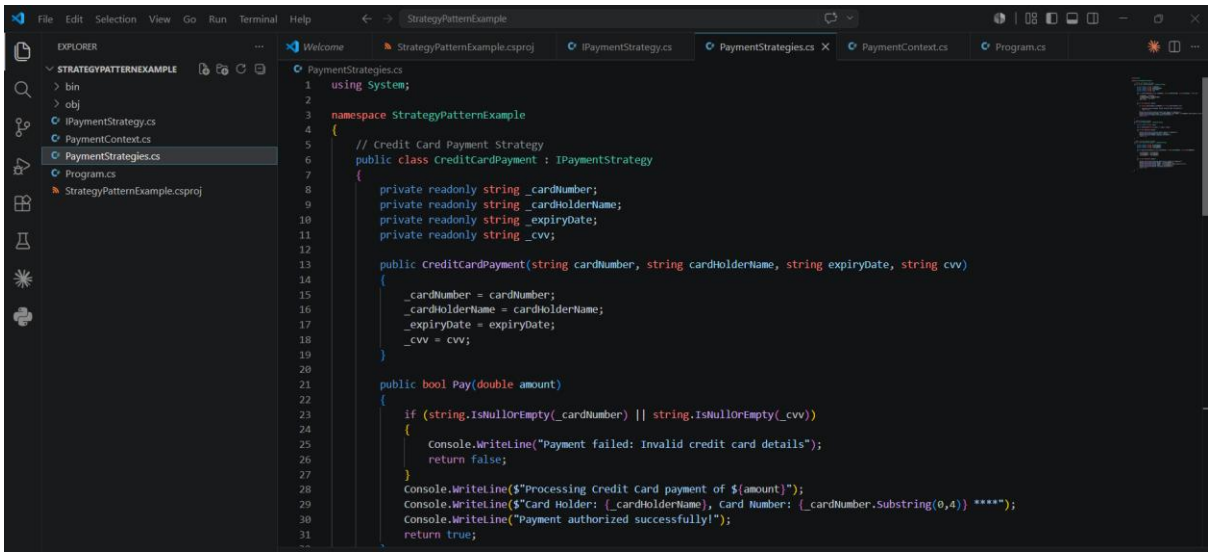
After running the application, I tested multiple payment methods by switching between strategies to verify that each payment process worked correctly. This hands-on activity helped me understand how the Strategy Pattern enables runtime selection of algorithms, improves flexibility, reduces complex conditional statements, and makes .NET applications easier to extend and maintain.

Coding:

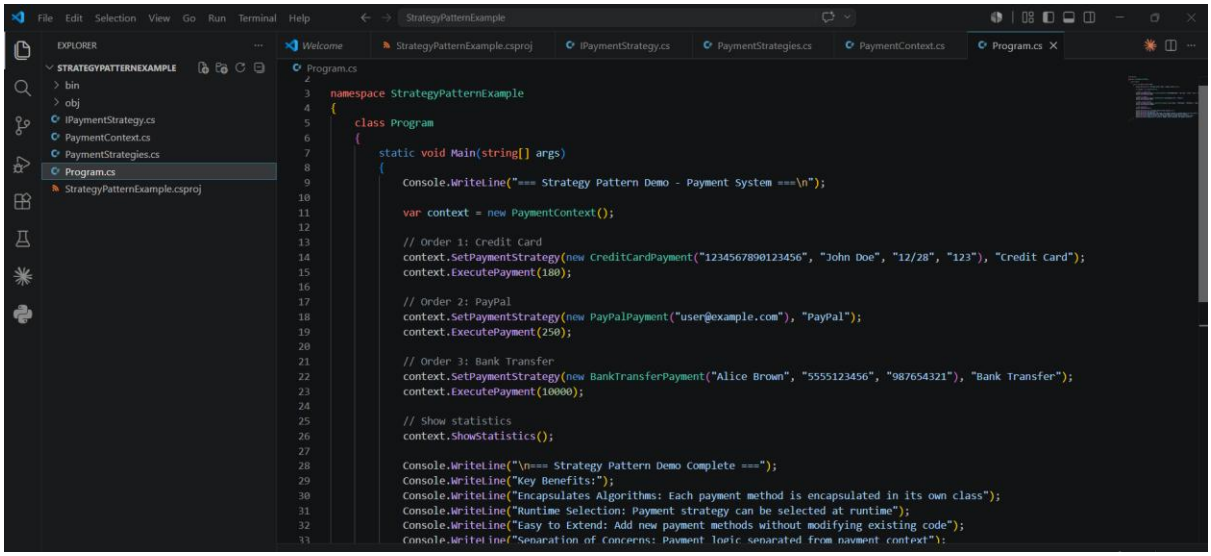




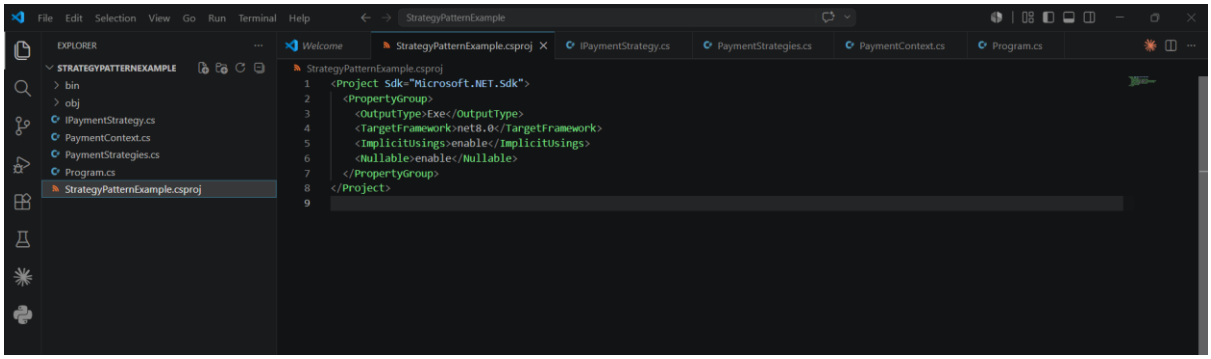
```
using System;

namespace StrategyPatternExample
{
    public class PaymentContext
    {
        private IPaymentStrategy? _paymentStrategy;
        private string _currentMethod = "None";
        private double _lastAmount;
        private DateTime _lastPaymentDate;

        public void SetPaymentStrategy(IPaymentStrategy strategy, string methodName)
        {
            _paymentStrategy = strategy;
            _currentMethod = methodName;
            Console.WriteLine($"\nPayment strategy set to: {methodName}");
        }

        public void ExecutePayment(double amount)
        {
            if (_paymentStrategy == null)
            {
                Console.WriteLine("No payment strategy selected!");
                return;
            }

            Console.WriteLine("\n=== Payment Processing ===");
            Console.WriteLine($"Amount: ${amount}");
            Console.WriteLine($"Method: {_currentMethod}");

            bool success = _paymentStrategy.Pay(amount);
```

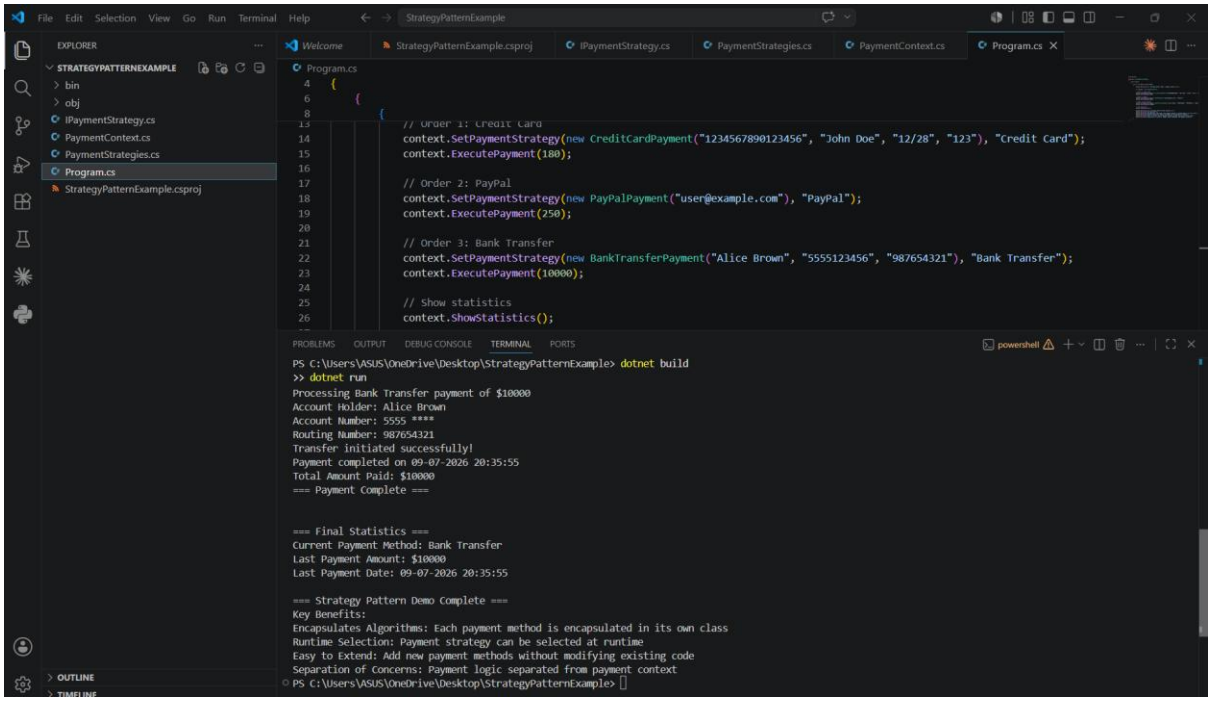


```
using System;

namespace StrategyPatternExample
{
    // Credit Card Payment Strategy
    public class CreditCardPayment : IPaymentStrategy
    {
        private readonly string _cardNumber;
        private readonly string _cardHolderName;
        private readonly string _expiryDate;
        private readonly string _cvv;

        public CreditCardPayment(string cardNumber, string cardHolderName, string expiryDate, string cvv)
        {
            _cardNumber = cardNumber;
            _cardHolderName = cardHolderName;
            _expiryDate = expiryDate;
            _cvv = cvv;
        }

        public bool Pay(double amount)
        {
            if (string.IsNullOrEmpty(_cardNumber) || string.IsNullOrEmpty(_cvv))
            {
                Console.WriteLine("Payment failed: Invalid credit card details");
                return false;
            }
            Console.WriteLine($"Processing Credit Card payment of ${amount}");
            Console.WriteLine($"Card Holder: {_cardHolderName}, Card Number: {_cardNumber.Substring(0,4)} ****");
            Console.WriteLine("Payment authorized successfully!");
            return true;
```



```
namespace StrategyPatternExample
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("=== Strategy Pattern Demo - Payment System ===\n");

            var context = new PaymentContext();

            // Order 1: Credit Card
            context.SetPaymentStrategy(new CreditCardPayment("1234567890123456", "John Doe", "12/28", "123"), "Credit Card");
            context.ExecutePayment(180);

            // Order 2: PayPal
            context.SetPaymentStrategy(new PayPalPayment("user@example.com"), "PayPal");
            context.ExecutePayment(250);

            // Order 3: Bank Transfer
            context.SetPaymentStrategy(new BankTransferPayment("Alice Brown", "5555123456", "987654321"), "Bank Transfer");
            context.ExecutePayment(10000);

            // Show statistics
            context.ShowStatistics();

            Console.WriteLine("\n=== Strategy Pattern Demo Complete ===");
            Console.WriteLine("Key Benefits:");
            Console.WriteLine("Encapsulates Algorithms: Each payment method is encapsulated in its own class");
            Console.WriteLine("Runtime Selection: Payment strategy can be selected at runtime");
            Console.WriteLine("Easy to Extend: Add new payment methods without modifying existing code");
            Console.WriteLine("Separation of Concerns: Payment logic separated from payment context");
```


File Edit Selection View Go Run Terminal Help
StrategyPatternExample
EXPLORER
STRATEGYPATTERNEXAMPLE
bin
obj
IPaymentStrategy.cs
PaymentContext.cs
PaymentStrategies.cs
Program.cs
StrategyPatternExample.csproj
Welcome
StrategyPatternExample.csproj
IPaymentStrategy.cs
PaymentStrategies.cs
PaymentContext.cs
Program.cs
StrategyPatternExample.csproj
<Project Sdk="Microsoft.NET.Sdk">
<PropertyGroup>
<OutputType>Exe</OutputType>
<TargetFramework>net8.0</TargetFramework>
<ImplicitUsings>enable</ImplicitUsings>
<Nullable>enable</Nullable>
</PropertyGroup>
</Project>


File Edit Selection View Go Run Terminal Help
StrategyPatternExample
EXPLORER
STRATEGYPATTERNEXAMPLE
bin
obj
IPaymentStrategy.cs
PaymentContext.cs
PaymentStrategies.cs
Program.cs
StrategyPatternExample.csproj
Welcome
StrategyPatternExample.csproj
IPaymentStrategy.cs
PaymentStrategies.cs
PaymentContext.cs
Program.cs
Program.cs
{
{
{
// Order 1: Credit Card
context.SetPaymentStrategy(new CreditCardPayment("1234567890123456", "John Doe", "12/28", "123"), "Credit Card");
context.ExecutePayment(180);
// Order 2: PayPal
context.SetPaymentStrategy(new PayPalPayment("user@example.com"), "PayPal");
context.ExecutePayment(250);
// Order 3: Bank Transfer
context.SetPaymentStrategy(new BankTransferPayment("Alice Brown", "5555123456", "987654321"), "Bank Transfer");
context.ExecutePayment(10000);
// Show statistics
context.ShowStatistics();
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
powershell
PS C:\Users\ASUS\OneDrive\Desktop\StrategyPatternExample> dotnet build
>> dotnet run
Processing Bank Transfer payment of $10000
Account Holder: Alice Brown
Account Number: 5555 ****
Routing Number: 987654321
Transfer initiated successfully!
Payment completed on 09-07-2026 20:35:55
Total Amount Paid: $10000
=== Payment Complete ===
=== Final Statistics ===
Current Payment Method: Bank Transfer
Last Payment Amount: $10000
Last Payment Date: 09-07-2026 20:35:55
=== Strategy Pattern Demo Complete ===
Key Benefits:
Encapsulates Algorithms: Each payment method is encapsulated in its own class
Runtime Selection: Payment strategy can be selected at runtime
Easy to Extend: Add new payment methods without modifying existing code
Separation of Concerns: Payment logic separated from payment context
PS C:\Users\ASUS\OneDrive\Desktop\StrategyPatternExample>
OUTLINE
TIMELINE